

Bancos de Dados NoSQL

Redis e MongoDB - Guia Completo

Agenda

1. **Redis: Cache e Estruturas Rápidas**
2. **MongoDB: NoSQL Orientado a Documentos**
3. **Comparativo entre Redis e MongoDB**
4. **Exemplos Práticos de Caso de Uso**
5. **Quando Usar Cada Um**
6. **Arquitetura Moderna com NoSQL**

Parte 1: Redis - Cache e Estruturas Rápidas

O que é Redis?

- **Tipo:** Banco de dados em memória (In-Memory)
- **Estrutura:** Chave-Valor (Key-Value Store)
- **Acesso:** Extremamente rápido (microsegundos)
- **Persistência:** Opcional (RDB, AOF)
- **Uso Principal:** Cache, filas, sessões, rankings

Redis (RAM)

Extremamente Rápido

Estruturas:

- String
- List
- Set
- Hash
- Sorted Set

Instalação e Configuração

Instalação (Linux/Mac)

```
# Ubuntu/Debian
sudo apt-get install redis-server

# macOS com Homebrew
brew install redis

# Iniciar Redis
redis-server

# Conectar ao cliente
redis-cli
```

Docker (Recomendado)

```
docker run -d -p 6379:6379 --name redis redis:latest  
  
# Cliente  
docker exec -it redis redis-cli
```

Tipos de Dados em Redis

1. String

```
SET usuario:1:nome "João"  
GET usuario:1:nome  
APPEND usuario:1:nome " Silva"  
STRLEN usuario:1:nome
```

Incremento/Decremento

```
SET contador 0  
INCR contador      # 1  
INCRBY contador 5  # 6  
DECR contador      # 5
```


2. List (Fila/Pilha)

```
# Adicionar
LPUSH tarefas "Tarefa 1"
RPUSH tarefas "Tarefa 2"
RPUSH tarefas "Tarefa 3"

# Visualizar
LRANGE tarefas 0 -1      # Todas as tarefas
LLEN tarefas             # Contar

# Remover
LPOP tarefas             # Remove da esquerda
RPOP tarefas             # Remove da direita
```

Tipos de Dados em Redis (Continuação)

3. Set (Conjunto Único)

```
# Adicionar
SADD amigos "Alice"
SADD amigos "Bob"
SADD amigos "Carlos"

# Visualizar
SMEMBERS amigos          # Listar todos

# Operações
SCARD amigos             # Contar
SISMEMBER amigos "Bob"   # Verificar se existe
SREM amigos "Bob"        # Remover
```

4. Hash (Objeto)

```
# Adicionar campos
HSET usuario:1 nome "João" email "joao@email.com" idade 30

# Visualizar
HGETALL usuario:1
HGET usuario:1 nome      # Pegar um campo
HKEYS usuario:1          # Todos os campos
HVALS usuario:1          # Todos os valores

# Modificar
HINCRBY usuario:1 idade 1 # Incrementar
HSET usuario:1 ativo true
```

5. Sorted Set (Conjunto Ordenado)

```
# Ranking de pontos
ZADD ranking 100 "Alice"
ZADD ranking 150 "Bob"
ZADD ranking 120 "Carlos"

# Visualizar (com scores)
ZRANGE ranking 0 -1 WITHSCORES
ZREVRANGE ranking 0 -1 WITHSCORES # Decrescente

# Operações
ZCARD ranking           # Contar
ZSCORE ranking "Bob"     # Score específico
ZRANK ranking "Alice"    # Posição (crescente)
ZREVRANK ranking "Alice" # Posição (decrescente)
```

TTL (Time To Live - Expiração)

Redis é perfeito para dados temporários:

```
# Definir com expiração
SET sessao:user123 "token_abc" EX 3600 # 1 hora

# Ou depois
SET sessao:user123 "token_abc"
EXPIRE sessao:user123 3600

# Verificar quanto tempo falta
TTL sessao:user123 # Segundos (-1 = sem expiração, -2 = não existe)

# Remover expiração
PERSIST sessao:user123

# Expiração em timestamp Unix
EXPIREAT senha:user123 1735689600
```

Exemplo Prático 1: Cache com Node.js

```
const redis = require("redis");
const client = redis.createClient({
  host: "localhost",
  port: 6379,
});

client.connect();

// Função com cache
async function buscarUsuario(id) {
  // Verificar cache
  const cache = await client.get(`usuario:${id}`);
  if (cache) {
    console.log("✓ Vindo do cache");
    return JSON.parse(cache);
  }

  // Se não tiver, buscar do BD
  console.log("✗ Consultando banco");
  const usuario = await consultarBancoDados(id);

  // Armazenar no cache por 1 hora
  await client.setEx(`usuario:${id}`, 3600, JSON.stringify(usuario));

  return usuario;
}

// Usar
await buscarUsuario(1); // Consulta BD
await buscarUsuario(1); // Vem do cache
```

Exemplo Prático 2: Contador de Visualizações

```
// Contar visualizações de um artigo
async function registrarVisualizacao(artigoId) {
  await client.incr(`artigo:${artigoId}:views`);

  // Fazer trending: adicionar ao ranking
  const views = await client.get(`artigo:${artigoId}:views`);
  await client.zadd("trending_artigos", views, artigoId);
}

// Obter trending
async function getTrending() {
  return await client.zrevrange("trending_artigos", 0, 4, "WITHSCORES");
}

// Usar
await registrarVisualizacao(1);
await getTrending(); // Top 5
```


Exemplo Prático 3: Fila de Tarefas

```
// Produtor: adicionar tarefa
async function enfileirarTarefa(dados) {
  await client.rpush("fila_emails", JSON.stringify(dados));
}

// Consumidor: processar tarefas
async function processarFilaTarefas() {
  while (true) {
    // Pega e remove da fila (bloqueante)
    const tarefa = await client.blpop("fila_emails", 0);

    if (tarefa) {
      const dados = JSON.parse(tarefa[1]);
      console.log("Processando:", dados);
      await enviarEmail(dados);
    }
  }
}

// Usar
await enfileirarTarefa({ email: "user@email.com", assunto: "Bem-vindo" });
processarFilaTarefas(); // Roda indefinidamente
```

Exemplo Prático 4: Sessões

```
// Armazenar sessão
async function criarSessao(userId) {
  const sessionId = crypto.randomUUID();

  await client.setEx(
    `sessao:${sessionId}`,
    86400 * 7, // 7 dias
    JSON.stringify({
      userId,
      loginAt: new Date(),
      permissions: ["read", "write"],
    })
  );

  return sessionId;
}

// Validar sessão
async function validarSessao(sessionId) {
  const sessao = await client.get(`sessao:${sessionId}`);
  return sessao ? JSON.parse(sessao) : null;
}

// Usar
const sessionId = await criarSessao(1);
const sessao = await validarSessao(sessionId); // ✓ Válida
```

Parte 2: MongoDB - NoSQL Flexível

O que é MongoDB?

- **Tipo:** Banco de Dados NoSQL Orientado a Documentos
- **Estrutura:** Coleções de Documentos JSON (BSON)
- **Esquema:** Flexível e Dinâmico
- **Escalabilidade:** Horizontal (Sharding)
- **Uso:** Dados não estruturados, prototipagem rápida, big data

MongoDB (Disco)

Estrutura:

- Database
 - Collection
 - Document (JSON)

Instalação e Configuração

Instalação Local (Linux/Mac)

```
# Ubuntu/Debian
sudo apt-get install -y mongodb-server

# macOS
brew tap mongodb/brew
brew install mongodb-community

# Iniciar
mongod
```

Docker (Recomendado)

```
docker run -d -p 27017:27017 --name mongodb mongo:latest  
  
# Cliente  
docker exec -it mongodb mongosh
```

MongoDB Cloud (Atlas)

```
https://www.mongodb.com/cloud/atlas  
  
# Connection String  
mongodb+srv://usuario:senha@cluster.mongodb.net/banco
```

Estrutura de um Documento

MongoDB armazena dados em documentos BSON (binary JSON):


```
// Exemplo de documento
{
  "_id": ObjectId("507f1f77bcf86cd799439011"),
  "nome": "João Silva",
  "email": "joao@email.com",
  "idade": 30,
  "ativo": true,
  "data_cadastro": ISODate("2024-05-18T10:30:00Z"),
  "endereço": {
    "rua": "Rua A",
    "cidade": "São Paulo",
    "cep": "01234-567"
  },
  "hobbies": ["leitura", "programação", "games"],
  "metadata": {
    "ultimoAcesso": ISODate("2024-05-18T15:45:00Z"),
    "tentativasLogin": 3
  }
}
```

Operações CRUD em MongoDB

CREATE (Inserir)

```
// Inserir um documento
db.usuarios.insertOne({
  nome: "João Silva",
  email: "joao@email.com",
  idade: 30,
});

// Inserir múltiplos documentos
db.usuarios.insertMany([
  { nome: "Maria", email: "maria@email.com", idade: 25 },
  { nome: "Pedro", email: "pedro@email.com", idade: 35 },
]);
```

READ (Consultar)

```
// Buscar todos
```

Operações CRUD (Continuação)

UPDATE (Atualizar)

```
// Atualizar um documento
db.usuarios.updateOne({ _id: ObjectId("...") }, { $set: { idade: 31 } });

// Atualizar múltiplos
db.usuarios.updateMany({ ativo: false }, { $set: { ativo: true } });

// Incrementar campo
db.usuarios.updateOne(
  { _id: ObjectId("...") },
  { $inc: { tentativas_login: 1 } },
);

// Adicionar a um array
db.usuarios.updateOne(
  { _id: ObjectId("...") },
  { $push: { hobbies: "culinária" } },
);
```

Operadores de Consulta

```
// Comparação
db.usuarios.find({ idade: { $eq: 30 } }); // Igual
db.usuarios.find({ idade: { $ne: 30 } }); // Não igual
db.usuarios.find({ idade: { $gt: 25 } }); // Maior que
db.usuarios.find({ idade: { $gte: 25 } }); // Maior ou igual
db.usuarios.find({ idade: { $lt: 35 } }); // Menor que
db.usuarios.find({ idade: { $lte: 35 } }); // Menor ou igual

// Arrays
db.usuarios.find({ hobbies: { $in: ["leitura", "games"] } });
db.usuarios.find({ hobbies: { $nin: ["esportes"] } });
db.usuarios.find({ hobbies: { $all: ["leitura", "programação"] } });
db.usuarios.find({ hobbies: { $size: 3 } }); // Array com 3 elementos

// Texto
db.usuarios.find({ nome: { $regex: "^Jo", $options: "i" } }); // Começa com Jo

// Lógica
db.usuarios.find({
  $and: [{ idade: { $gte: 25 } }, { ativo: true }],
});

db.usuarios.find({
  $or: [{ idade: { $lt: 20 } }, { idade: { $gt: 60 } }],
});
```

Agregação em MongoDB

Pipeline de transformação de dados:

```
db.usuarios.aggregate([
  // Stage 1: Filtrar
  {
    $match: { ativo: true },
  },
  // Stage 2: Agrupar
  {
    $group: {
      _id: null,
      totalUsuarios: { $sum: 1 },
      idadeMedia: { $avg: "$idade" },
      usuarioMaisVelho: { $max: "$idade" },
    },
  },
]);
```

```
// Resultado: { totalUsuarios: 3, idadeMedia: 30, usuarioMaisVelho: 35 }
```

Índices em MongoDB

Otimizam as consultas:

```
// Criar índice simples
db.usuarios.createIndex({ email: 1 });

// Índice composto
db.usuarios.createIndex({ ativo: 1, data_cadastro: -1 });

// Índice único
db.usuarios.createIndex({ email: 1 }, { unique: true });

// Índice de texto (busca full-text)
db.usuarios.createIndex({ nome: "text", email: "text" });
db.usuarios.find({ $text: { $search: "João" } });

// Listar índices
db.usuarios.getIndexes();

// Remover índice
db.usuarios.dropIndex("email_1");
```

Validação de Schema em MongoDB

MongoDB permite validação de documentos:

```
db.createCollection("usuarios", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["nome", "email"],
      properties: {
        _id: { bsonType: "objectId" },
        nome: {
          bsonType: "string",
          description: "Nome é obrigatório",
        },
        email: {
          bsonType: "string",
          pattern: "^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$",
        },
        idade: {
          bsonType: "int",
          minimum: 0,
          maximum: 150,
        },
      },
    },
  },
})
```

Exemplo Prático 1: CRUD com Node.js

```
const { MongoClient } = require("mongodb");

const client = new MongoClient("mongodb://localhost:27017");

async function exemplosCRUD() {
  try {
    await client.connect();
    const db = client.db("meu_app");
    const usuarios = db.collection("usuarios");

    // CREATE
    const resultado = await usuarios.insertOne({
      nome: "João",
      email: "joao@email.com",
      createdAt: new Date(),
    });
    console.log("Inserido:", resultado.insertedId);

    // READ
    const usuario = await usuarios.findOne({ email: "joao@email.com" });
    console.log("Encontrado:", usuario);

    // UPDATE
    await usuarios.updateOne(
      { _id: resultado.insertedId },
      { $set: { idade: 30 } },
    );

    // DELETE
    await usuarios.deleteOne({ _id: resultado.insertedId });
  } finally {
    await client.close();
  }
}

exemplosCRUD();
```


Exemplo Prático 2: Busca e Paginação

```
async function buscarUsuarios(pagina = 1, limite = 10) {
  const usuarios = db.collection("usuarios");

  const skip = (pagina - 1) * limite;

  const dados = await usuarios
    .find({ ativo: true })
    .sort({ data_cadastro: -1 })
    .skip(skip)
    .limit(limite)
    .toArray();

  const total = await usuarios.countDocuments({ ativo: true });

  return {
    dados,
    total,
    pagina,
    totalPaginas: Math.ceil(total / limite),
  };
}

// Usar
const resultado = await buscarUsuarios(2, 20);
console.log(resultado);
// { dados: [...], total: 150, pagina: 2, totalPaginas: 8 }
```

Exemplo Prático 3: Agregação Avançada

```
// Relatório: Usuários por cidade e idade média
async function relatorioUsuariosComPaginação() {
  return await db
    .collection("usuarios")
    .aggregate([
      {
        $match: { ativo: true },
      },
      {
        $group: {
          _id: "$endereco.cidade",
          totalUsuarios: { $sum: 1 },
          idadeMedia: { $avg: "$idade" },
          emails: { $push: "$email" },
        },
      },
      {
        $sort: { totalUsuarios: -1 },
      },
      {
        $limit: 10,
      },
    ])
    .toArray();
}

// Resultado:
// [
//   { _id: "São Paulo", totalUsuarios: 45, idadeMedia: 28.5, emails: [...] },
//   { _id: "Rio de Janeiro", totalUsuarios: 32, idadeMedia: 31.2, emails: [...] }
// ]
```

Exemplo Prático 4: Busca em Tempo Real

```
// Criar índice de texto
db.collection("posts").createIndex({ titulo: "text", conteudo: "text" });

// Buscar posts
async function buscarPosts(termo) {
  return await db
    .collection("posts")
    .find({
      $text: { $search: termo },
    })
    .project({
      score: { $meta: "textScore" },
    })
    .sort({
      score: { $meta: "textScore" },
    })
    .toArray();
}

// Usar
await buscarPosts("MongoDB Node.js");
```

Parte 4: Comparativo

Tabela Comparativa

Aspecto	Redis	MongoDB
Tipo	In-Memory KV	NoSQL Documento
Armazenamento	RAM	Disco
Velocidade	Muito Rápida	Rápida
Esquema	Sem esquema	Flexível
Transações	Limitadas	Suporta
Escalabilidade	Horizontal	Horizontal
Persistência	Opcional	Nativa
Caso de Uso	Cache, Sessões	Documentos

Quando Usar Redis

✓ Use Redis quando:

- Precisa de cache ultrarrápido
- Gerenciando sessões de usuário
- Processando filas de tarefas
- Criando rankings em tempo real
- Rate limiting
- Chat em tempo real
- Contadores e estatísticas

✗ Evite Redis quando:

- Os dados precisam ser permanentes sem sincronização
- Fazendo consultas complexas com múltiplos critérios

Quando Usar MongoDB

✓ Use MongoDB quando:

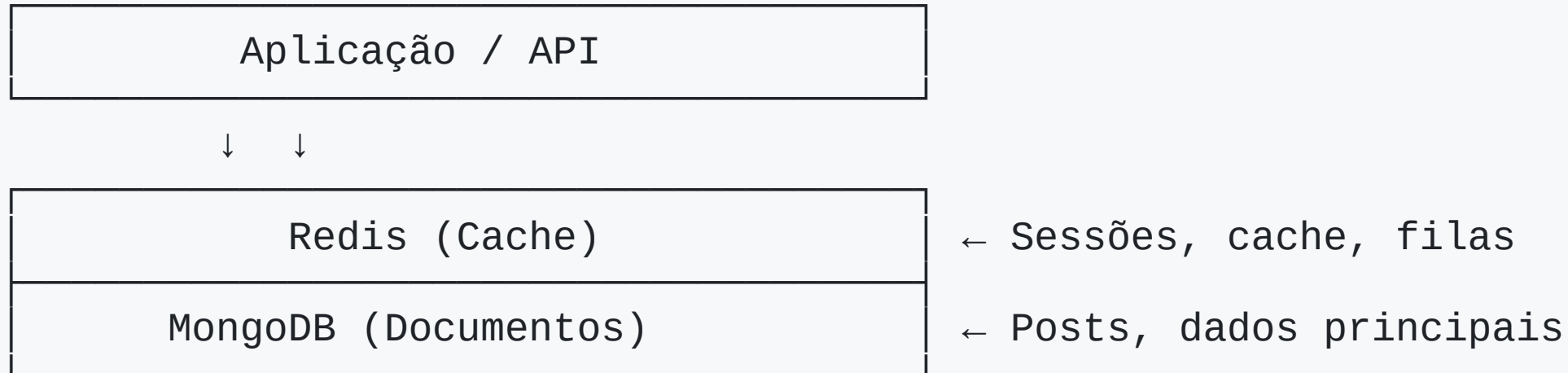
- O esquema é dinâmico/evolui rapidamente
- Os documentos têm formatos variados
- Prototipando rapidamente
- Armazenando dados semi-estruturados
- Precisa escalar horizontalmente
- Dados flexíveis com agregações complexas

✗ Evite MongoDB quando:

- Os dados têm muitos relacionamentos complexos
- Precisa de consistência ACID transacional complexa
- Os dados são muito simples e estruturados

Exemplos: CMS, redes sociais, IoT, logs de aplicação, dashboards

Arquitetura Moderna: Redis + MongoDB



Fluxo de Dados

1. Usuário faz requisição
2. Verifica cache no Redis
3. Se não estiver, consulta MongoDB
4. Armazena resultado no Redis com TTL

Parte 3: Exemplos de Caso de Uso Completo

Caso 1: E-commerce com Catálogo Flexível

Estrutura

```
// Redis: Cache e sessão
SET session:user123 JSON(sessaoData) EX 3600
ZADD trending_produtos 100 "Produto A"
ZADD trending_produtos 150 "Produto B"
INCR produto:123:views

// MongoDB: Catálogo com specs variáveis
db.produtos.insertOne({
  nome: "Notebook",
  preco: 3000,
  categoria: "Eletrônicos",
  specs: {
    cpu: "i7",
    ram: "16GB",
    ssd: "512GB"
  },
  reviews: [
    { usuario: "João", nota: 5, comentario: "Excelente!" }
  ],
  disponivel: true,
  ativo: true
})
```

Caso 2: Rede Social com Feeds em Tempo Real

Estrutura

```
// Redis: Feeds em tempo real
LPUSH feed:user123 postId_456
LPUSH feed:user123 postId_789
LRANGE feed:user123 0 19 // Últimos 20 posts
```

```
// MongoDB: Posts flexible
db.posts.insertOne({
  usuario_id: 123,
  conteudo: "Olá mundo!",
  midia: ["img1.jpg", "img2.jpg"],
  likes: 45,
  comentarios: [
    { usuario: 456, texto: "Ótimo post!" }
  ],
  createdAt: new Date()
})
```

Caso 3: Sistema de Analytics

Estrutura

```
// Redis: Contadores de eventos
INCR evento:pageview:dia:2024-05-18
INCR evento:click:dia:2024-05-18

// MongoDB: Análises detalhadas
db.eventos.insertOne({
  app_id: 1,
  tipo: "pageview",
  usuario_id: 123,
  pagina: "/blog/post1",
  timestamp: new Date(),
  metadata: {
    browser: "Chrome",
    so: "Linux",
    duracao_ms: 5000
  }
})
```

Código Completo: App com Redis e MongoDB

```
// app.js - Express com Redis e MongoDB
const express = require("express");
const redis = require("redis");
const { MongoClient } = require("mongodb");

const app = express();
const redisClient = redis.createClient();
const mongoClient = new MongoClient("mongodb://localhost:27017");

redisClient.connect();

// Middleware: Cache
async function cache(req, res, next) {
  const chave = `req:${req.originalUrl}`;
  const cached = await redisClient.get(chave);

  if (cached) {
    return res.json(JSON.parse(cached));
  }

  res.json = (function (json) {
    return function (dados) {
      redisClient.setEx(chave, 300, JSON.stringify(dados));
      return json.call(this, dados);
    };
  })(res.json);

  next();
}

// Rota: Listar posts (MongoDB)
app.get("/posts", cache, async (req, res) => {
  const db = mongoClient.db("meu_app");
  const posts = await db.collection("posts").find({}).limit(20).toArray();
  res.json(posts);
});

// Rota: Trending (Redis)
app.get("/trending", async (req, res) => {
  const trending = await redisClient.zrevrange(
    "trending_posts",
    0,
    9,
    "WITHSCORES",
  );
  res.json(trending);
});

// Rota: Registrar visualização
app.post("/posts/:id/view", async (req, res) => {
  const { id } = req.params;

  // Incrementar contador no Redis
  await redisClient.incr(`post:${id}:views`);

  // Atualizar ranking
  const views = await redisClient.get(`post:${id}:views`);
  await redisClient.zadd("trending_posts", views, id);

  // Registrar no MongoDB
  const db = mongoClient.db("meu_app");
  await db.collection("eventos").insertOne({
    tipo: "visualizacao",
    post_id: parseInt(id),
    timestamp: new Date(),
  });

  res.json({ sucesso: true });
});

app.listen(3000, () => {
  console.log("Servidor rodando em http://localhost:3000");
});
```

Dicas de Performance

1. Indexação

```
// MongoDB
db.usuarios.createIndex({ email: 1 });

// Redis - usar estruturas apropriadas
ZADD ranking score membro // Sorted Set para rankings
```

2. Pagination

```
// MongoDB
db.usuarios.find().skip(0).limit(20);

// Redis - usando sorted sets
ZRANGE key 0 19 // Primeiros 20 elementos
```

3. Projeção (Selecionar apenas o necessário)

Monitoramento e Debugging

Redis

```
# Info geral
redis-cli INFO

# Monitorar comandos em tempo real
redis-cli MONITOR

# Analisar memória
redis-cli INFO MEMORY
```

MongoDB

```
// Analisar query
db.usuarios.find({ email: "test@email.com" }).explain("executionStats");

// Monitorar performance
db.setProfilingLevel(1, { slowms: 100 });
```


Boas Práticas Finais

1. Sempre usar índices nas buscas frequentes

- MongoDB: `createIndex()`
- Redis: Estruturas apropriadas (Sorted Sets, Sets, etc)

2. Limpar dados antigos (TTL)

```
// MongoDB
db.collection.createIndex({ createdAt: 1 }, { expireAfterSeconds: 86400 });

// Redis
EXPIRE chave 3600
```

3. Backups regulares

```
# MongoDB
mongodump --uri="mongodb://..."

# Redis
BGSAVE
```

4. Monitorar alertas

- Espaço em disco
- Uso de memória (especialmente Redis)
- Conexões ativas
- Queries lentas (MongoDB)

5. Documentar o schema

```
// README.md com exemplo de documento MongoDB
// Índices criados
```

Conclusão

- **Redis:** Para cache ultrarrápido, sessões e estruturas em memória
- **MongoDB:** Para dados flexíveis, semiestruturados e escaláveis

A melhor escolha é usar TODOS em arquitetura moderna!

```
Redis + MongoDB = NoSQL Stack|
```

Recursos Adicionais

Documentação Oficial:

- Redis: <https://redis.io/documentation>
- MongoDB: <https://docs.mongodb.com/>

Cursos Online:

- The Complete Redis Course (Udemy)
- MongoDB Masterclass (Udemy)
- Redis Deep Dive (Pluralsight)

Ferramentas Úteis:

- MongoDB Compass (MongoDB GUI)
- Redis Desktop Manager (Redis GUI)

Redis Insight (Redis GUI - Monitoring)

Próximas Aulas

1. **Redis Avançado** - Pub/Sub, Streams, Lua Scripts
2. **MongoDB Agregações** - Pipeline complexo
3. **Segurança em NoSQL** - Autenticação e criptografia
4. **Docker Compose** - Orquestração de serviços
5. **Casos de Uso Avançados** - Real-world applications em produção

Perguntas?